

Análisis de Patrones de Diseño — Sistema Biblioteca

2. ANÁLISIS DE PATRONES — Sistema de Biblioteca

El siguiente análisis detalla los tres patrones de diseño GoF implementados en el Sistema de Biblioteca, evaluando el problema identificado en el contexto real del proyecto, las alternativas consideradas, el patrón seleccionado, los beneficios obtenidos y su representación mediante diagrama UML.

2.1 Patrón Singleton — Conn.java

Problema identificado

El sistema cuenta con cinco DAOs (`DAOSocio` , `DAOAutor` , `DAOCategoria` , `DAOLibro` , `DAOPrestamo`), todos los cuales requieren acceso a la base de datos MySQL para ejecutar sus consultas. Sin un mecanismo de control, cada operación que un formulario Swing invoca sobre un DAO podría abrir una nueva conexión física TCP/IP con el servidor MySQL y cerrarla al terminar. Dado que los formularios realizan múltiples consultas seguidas (listar socios, listar libros, registrar préstamo), este patrón de apertura y cierre repetido por operación genera una sobrecarga significativa: latencia en cada formulación de conexión, consumo innecesario de hilos del servidor y riesgo de superar el límite de conexiones concurrentes de MySQL.

Alternativas posibles

Alternativa	Descripción	Por qué no fue seleccionada
Conexión directa por DAO	Cada DAO crea y cierra su conexión en cada método.	Genera N aperturas de conexión por operación compuesta, degradando el rendimiento y saturando el servidor.
Connection Pool (HikariCP / C3P0)	Pool de múltiples conexiones reutilizables gestionado por una librería externa.	Introduce dependencias externas innecesarias para una aplicación de escritorio mono-usuario de uso secuencial. Agrega complejidad de configuración sin beneficio proporcional al contexto.
Singleton	Una única instancia de <code>Connection</code> compartida por todos los DAOs para toda la sesión.	Seleccionado: máximo rendimiento con mínima complejidad para el contexto de escritorio mono-usuario.

Patrón seleccionado

Singleton (GoF — Creacional)

`Conn.java` implementa el Singleton mediante:

- Constructor **privado** que impide la instanciación directa desde cualquier clase externa.
- Atributo estático `instancia` de tipo `Connection`.
- Método estático `get()` que retorna siempre la misma instancia, o la recrea automáticamente si fue cerrada (`isClosed()`), garantizando resiliencia ante desconexiones inesperadas.

```

public static Connection get() throws SQLException {
    if (instancia == null || instancia.isClosed()) {
        instancia = DriverManager.getConnection(URL, USUARIO, PASSWORD);
    }
    return instancia;
}

```

Beneficios obtenidos

- **Rendimiento:** Una sola conexión física activa elimina el overhead de apertura/cierre repetido de sockets TCP con MySQL.
- **Centralización:** La cadena de conexión (`URL` , `USUARIO` , `PASSWORD`) está definida en un único lugar; cualquier cambio de entorno se realiza en una sola clase.
- **Transparencia para los DAOs:** Todos los DAOs usan `Conn.get()` sin conocer los detalles de configuración de la conexión.
- **Resiliencia:** La verificación `isClosed()` permite reconexión automática sin intervención del usuario.
- **Adecuación al contexto:** Para una aplicación de escritorio mono-usuario, el Singleton provee el mismo beneficio de reutilización que un pool sin su complejidad asociada.

Diagrama UML

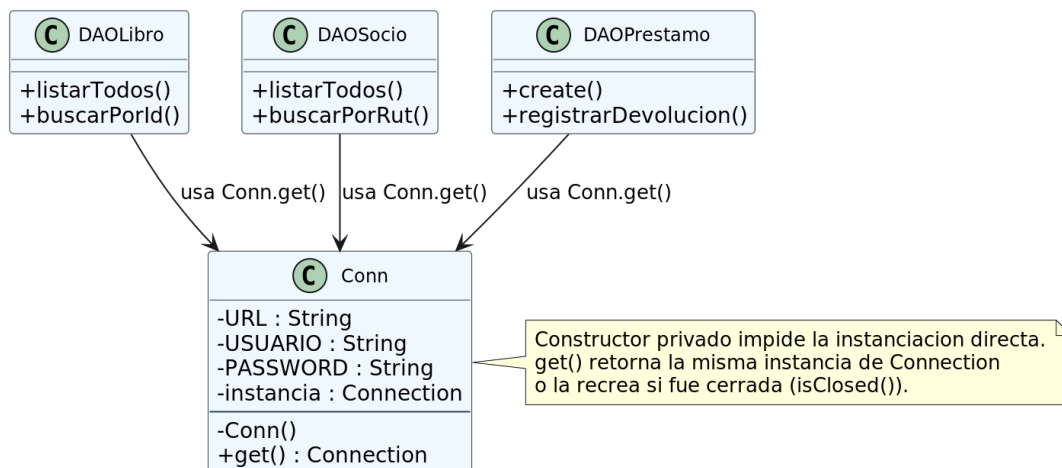


Diagrama Singleton

2.2 Patrón Facade — BibliotecaFacade.java

Problema identificado

El proyecto cuenta con cinco DAOs especializados y varias reglas de negocio críticas distribuidas entre ellos. Sin un mediador, cada formulario Swing de la capa GUI debería instanciar y coordinar directamente múltiples DAOs para completar una sola operación. Por ejemplo, para registrar un préstamo, `FrmPrestamo` debería: instanciar `DAOSocio` para verificar que el socio esté activo, instanciar `DAOLibro` para verificar el stock disponible, e instanciar `DAOPrestamo` para insertar el registro. Este acoplamiento directo GUI → DAOs genera un código difícil de mantener, con reglas de validación duplicadas o dispersas, y hace que cambiar la lógica de negocio requiera modificar múltiples formularios.

Alternativas posibles

Alternativa	Descripción	Por qué no fue seleccionada
Acceso directo a DAOs	Los formularios Swing instancian los DAOs directamente.	Alto acoplamiento entre capas. Duplicación de reglas de validación. Cambios en la capa de datos impactan en todos los formularios.
Service Layer (sin Facade)	Servicios separados por entidad (SocioService, LibroService).	No resuelve la coordinación multi-DAO; la GUI aún debe orquestrar múltiples servicios para operaciones compuestas como el préstamo.
Facade	Una única clase que encapsula todos los DAOs y expone métodos de negocio de alto nivel.	Seleccionado: desacopla completamente la GUI de la persistencia, centraliza las validaciones y permite testear la lógica sin Swing.

Patrón seleccionado

Facade (GoF — Estructural)

`BibliotecaFacade.java` implementa la Fachada mediante:

- **Encapsulación de los cinco DAOs** como atributos privados de la clase.
- **Métodos de alto nivel** que orquestan múltiples DAOs con validaciones de negocio integradas.
- **Instancia única** (combinación con Singleton mediante `getInstancia()`) para que todos los formularios compartan el mismo sujeto observable.

```
public void registrarPrestamo(Prestamo p) {
    // Validación 1: stock del libro
    Libro libro = daoLibro.buscarPorId(p.getLibro().getIdLibro());
    if (libro == null || libro.getCantidadDisponible() <= 0)
        throw new IllegalStateException("Sin ejemplares disponibles.");

    // Validación 2: estado del socio
    Socio socio = daoSocio.buscarPorId(p.getSocio().getIdSocio());
    if (socio == null || !"Activo".equalsIgnoreCase(socio.getEstado()))
        throw new IllegalStateException("Socio no activo.");

    daoPrestamo.create(p);
    notificarPrestamoRegistrado(p); // dispara el Observer
}
```

Beneficios obtenidos

- **Desacoplamiento total:** Los formularios (`FrmPrestamo`, `FrmBusquedaLibro`, `FrmLibro`, `FrmSocio`) solo dependen de `BibliotecaFacade`; ninguno importa clases DAO directamente.
- **Centralización de reglas de negocio:** Las validaciones críticas (stock, estado del socio, cálculo de multa) viven en un único punto, eliminando el riesgo de inconsistencias.
- **Testeabilidad:** `BibliotecaFacadeTest.java` puede verificar la lógica de negocio de forma aislada sin necesidad de componentes Swing.
- **Composición natural con Observer:** La Fachada actúa simultáneamente como Subject del patrón Observer, coordinando la notificación a los formularios registrados.
- **Extensibilidad:** Agregar una nueva entidad (ej.: `Reserva`) requiere añadir un DAO y métodos a la Fachada sin modificar los formularios existentes.

Diagrama UML

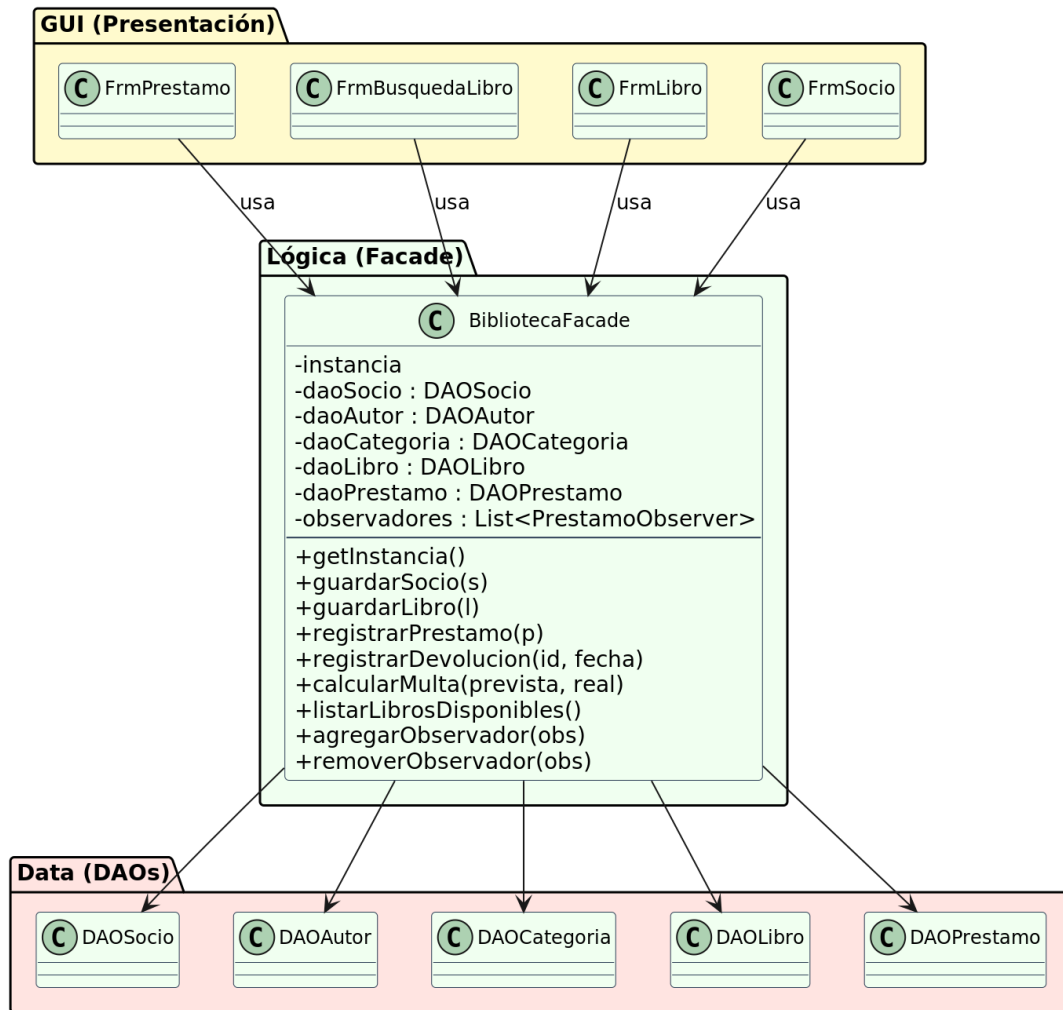


Diagrama Facade

2.3 Patrón Observer — `PrestamoObserver.java`

Problema identificado

`FrmPrestamo` y `FrmBusquedaLibro` son dos `JInternalFrame` que pueden estar abiertas simultáneamente en el escritorio de la aplicación. Cuando el usuario registra un préstamo en `FrmPrestamo`, el stock del libro en la base de datos disminuye en una unidad. Sin un mecanismo de notificación, `FrmBusquedaLibro` continuaría mostrando el dato de disponibilidad anterior (desactualizado) hasta que el usuario realice una búsqueda manual. El problema se agudiza con las devoluciones: el stock aumenta,

pero `FrmPrestamo` tampoco refrescaría su combo de libros disponibles. Un acoplamiento directo entre formularios (ej.: que `FrmPrestamo` guarde una referencia a `FrmBusquedaLibro` para llamarla directamente) generaría dependencias frágiles, difíciles de escalar y contrarias al principio de responsabilidad única.

Alternativas posibles

Alternativa	Descripción	Por qué no fue seleccionada
Polling con Timer	Un <code>javax.swing.Timer</code> reconsulta la BD cada N segundos para refrescar las vistas.	Introduce latencia artificial y consultas innecesarias a la BD incluso cuando no hay cambios. Consume recursos de red y CPU de forma continua.
Referencia directa entre formularios	<code>FrmPrestamo</code> guarda una referencia explícita a <code>FrmBusquedaLibro</code> y la notifica directamente.	Alto acoplamiento entre formularios de presentación. No es escalable: cada nuevo formulario requeriría modificar <code>FrmPrestamo</code> . Viola el principio Abierto/Cerrado.
EventBus / Bus de eventos	Un bus central de eventos desacoplado (ej.: Guava EventBus).	Introduce una dependencia externa innecesaria; el patrón Observer nativo de Java provee la misma funcionalidad sin librerías adicionales para este contexto.
Observer	<code>BibliotecaFacade</code> actúa como Subject; los formularios se suscriben a eventos de préstamo.	Seleccionado: máximo desacoplamiento, reactividad instantánea y sin dependencias externas.

Patrón seleccionado

Observer (GoF — Comportamiento)

La implementación del patrón se articula en tres elementos:

1. **Interfaz `PrestamoObserver`** — Define el contrato que deben cumplir todos los observadores, con dos eventos: `onPrestamoRegistrado(Prestamo p)` y `onPrestamoDevuelto(Prestamo p)`.
2. **Subject: `BibliotecaFacade`** — Mantiene la lista de observadores suscritos, y los notifica automáticamente al final de `registrarPrestamo()` y `registrarDevolucion()`.
3. **Observadores concretos: `FrmPrestamo` y `FrmBusquedaLibro`** — Implementan `PrestamoObserver`. Se suscriben en el constructor y se desuscriben en el método `dispose()`, gestionando correctamente su propio ciclo de vida.

```
// FrmPrestamo - constructor
BibliotecaFacade.getInstance().agregarObservador(this);

// FrmPrestamo - respuesta al evento
@Override
public void onPrestamoRegistrado(Prestamo p) {
    actualizarCombosYTablas(); // refresca combos y tabla de activos
}

// FrmPrestamo - limpieza al cerrar
@Override
public void dispose() {
    BibliotecaFacade.getInstance().removeObservador(this);
    super.dispose();
}
```

Beneficios obtenidos

- **Reactividad en tiempo real:** `FrmBusquedaLibro` actualiza la tabla de libros (incluyendo la disponibilidad) de forma instantánea al registrar o devolver un préstamo, sin intervención manual del usuario.
- **Desacoplamiento total entre formularios:** `BibliotecaFacade` notifica a una lista abstracta de `PrestamoObserver` sin conocer las clases concretas `FrmPrestamo` ni `FrmBusquedaLibro`.
- **Extensibilidad (Open/Closed Principle):** Agregar un nuevo formulario que necesite reaccionar a eventos de préstamo solo requiere que implemente `PrestamoObserver` y se suscriba; no se modifica ninguna clase existente.

- **Gestión del ciclo de vida:** La desuscripción en `dispose()` previene memory leaks al evitar que la Fachada mantenga referencias a formularios ya cerrados.
- **Eliminación de consultas redundantes:** Solo se consulta la BD cuando efectivamente ocurre un cambio de estado (evento), a diferencia del polling que consulta de forma continua.

Diagrama UML

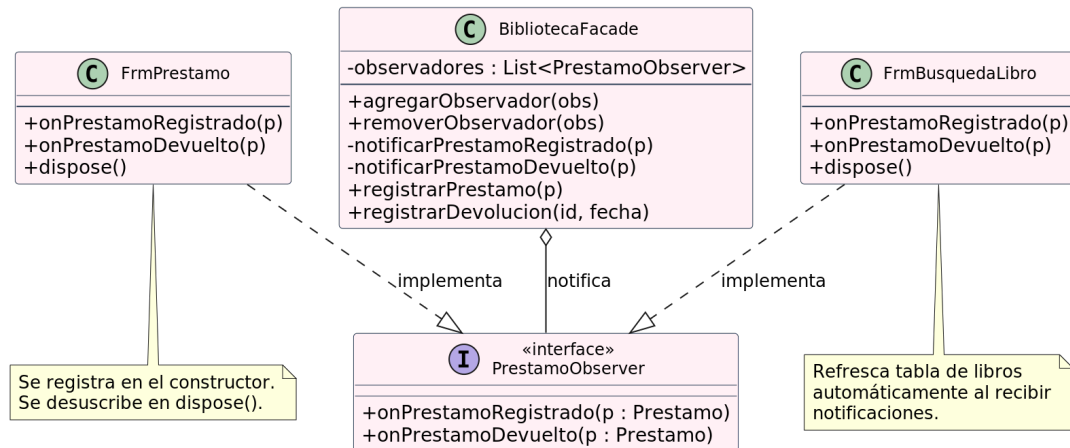


Diagrama Observer

Análisis elaborado con base en el código fuente del repositorio: github.com/Salmon5g/

SistemaBiblioteca